МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ

(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

ОТЧЕТ

по лабораторной работе № 2
по дисциплине «Операционные системы»

Тема: Управление процессами и межпроцессное взаимодействие (Pipes) в POSIX

Выполнил:
студент группы М8О-203БВ-25
Эль-Тахир Роман Райидович

Проверил:
преподаватель
Миронов Е.С.

Москва – 2026

1. Цель работы

Изучить механизмы создания процессов и межпроцессного взаимодействия в операционных системах Linux на примере неименованных каналов pipe, перенаправление стандартных потоков ввода-вывода, а также организацию кода с отделением работы с системными вызовами от общей логики программы.(`fork`, `exec`, `waitpid`) и организации межпроцессного взаимодействия с использованием неименованных каналов (`pipe`).

2. Постановка задачи

Необходимо разработать две **отдельные** программы: родительскую (`parent`) и дочернюю (`child`).

- Родительский процесс запрашивает у пользователя два имени файлов: первая введённая строка — файл для `child1`, вторая — для `child2`.
- Родитель создаёт два неименованных канала `pipe1` и `pipe2`, открывает выходные файлы и запускает два дочерних процесса.
- Все последующие строки, введённые в консоль, распределяются по каналам: **нечётные** строки (1-я, 3-я, ...) отправляются в `pipe1` (для `child1`), **чётные** (2-я, 4-я, ...) — в `pipe2` (для `child2`).
- Дочерние процессы читают строки из своего канала через `stdin`, **инвертируют** каждую строку и выводят в `stdout`, который родитель перенаправил в соответствующий файл.
- Программа должна работать в Linux (POSIX).

Пример. Ввод: `out1.txt`, `out2.txt`, `abc`, `def`, `ghi`, `jkl`. Результат: в `out1.txt` — `cba`, `ihg`; в `out2.txt` — `fed`, `lkj`.

3. Структура проекта и архитектура

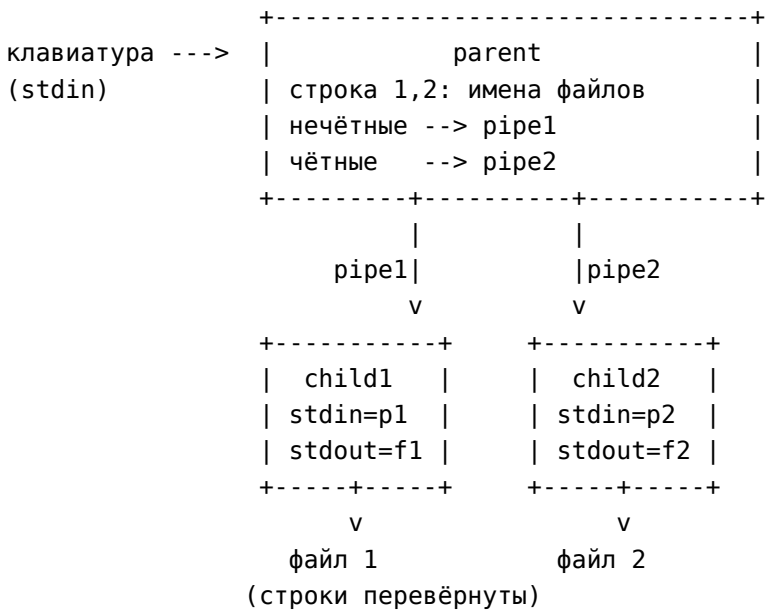
Проект собирается системой CMake и состоит из следующих модулей:

Файл / папка	Назначение
`common/errors.c`	Вывод сообщений об ошибках (`errno`).
`common/io_utils.c`	Чтение строки произвольной длины (`read_line`).
`os/os.h`	Единый интерфейс работы с процессами, каналами и файлами.
`os/os_linux.c`	Реализация интерфейса для Linux (POSIX).
`parent/main.c`	Родительская программа.
`child/main.c`	Дочерняя программа (инверсия строк).
`tests/run_test.cmake`	Автоматический тест (`ctest`).

Ключевая идея архитектуры — **слой абстракции**. Файл `os.h` объявляет набор функций (`os_pipe_create`, `os_process_start`, `os_write_all`, `os_close`, `os_process_wait`...) и типы данных. Логика родителя в `parent/main.c` не содержит прямых вызовов `fork` или `execl`: они скрыты внутри `os_linux.c`. Типы вынесены в `os.h`:

```
typedef int os_handle_t;            /* Linux: fd (int)   */
#define OS_INVALID_HANDLE (-1)
```

3.1. Схема взаимодействия процессов



4. Описание основных системных вызовов (Linux)

4.1. Директива `#define _GNU_SOURCE`

Макрос, который необходимо указать **до** всех `#include`. Он включает в заголовочных файлах библиотеки glibc нестандартные расширения Linux. В нашем коде он нужен для функции `pipe2()`: без него её объявления в `<unistd.h>` не будет.

4.2. Тип `os_handle_t`

Собственный тип проекта. Это `int` — файловый дескриптор, то есть номер открытого файла, канала или сокета в таблице процесса. Отдельное имя типа делает интерфейс `os.h` понятным и позволяет при необходимости изменить представление дескриптора, не меняя код родителя.

4.3. Тип `ssize_t`

Знаковый целочисленный тип для размеров. Отличается от `size_t` наличием знака, что необходимо, так как функции `read`, `write`, `readlink` возвращают число байт либо `-1` при ошибке.

4.4. Функция `pipe2`

```
int pipe2(int fds[2], int flags);
```

Создаёт неименованный канал — однонаправленный буфер в ядре. В `fds[0]` возвращается дескриптор для чтения, в `fds[1]` — для записи. Данные, записанные в `fds[1]`, можно прочитать из `fds[0]` в порядке FIFO. В отличие от обычной `pipe`, функция `pipe2` принимает флаги и позволяет атомарно установить `O_CLOEXEC`. Это важно в многопоточных программах, где между `pipe` и `fcntl` мог бы произойти `fork+exec`.

4.5. Функция `open` и её флаги

```
int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC | O_CLOEXEC, 0644);
```

Флаги объединяются побитовым ИЛИ:

- `O_WRONLY` — открыть файл только на запись;
- `O_CREAT` — создать файл, если он не существует;
- `O_TRUNC` — если файл существует, обнулить его содержимое;
- `O_CLOEXEC` — закрыть дескриптор автоматически при вызове `exec`.

Число `0644` (восьмеричное) — права нового файла: владелец читает и пишет, остальные только читают.

4.6. Флаг `O_CLOEXEC`

Флаг «close on exec». Без него дочерняя программа после `exec` унаследовала бы *все* открытые дескрипторы родителя, в том числе записывающие концы каналов. Тогда `child` никогда не получил бы конец файла (EOF) на `stdin`, потому что у него самого оставалась бы открытая копия записывающего конца, и программа зависла бы. Благодаря `O_CLOEXEC` после `exec` остаются только дескрипторы 0, 1, 2, явно установленные через `dup2`.

4.7. Функция `fork`

Создаёт копию текущего процесса. В родителе возвращает идентификатор (pid) ребёнка, в ребёнке — `0`, при ошибке — `-1`. Это позволяет в одном коде развести поведение родителя и ребёнка:

```
pid_t pid = fork();
if (pid < 0)  { /* ошибка */ }
if (pid == 0) { /* код ребёнка */ }
proc->pid = pid; /* код родителя */
```

4.8. Функция `dup2`

```
dup2(in,  STDIN_FILENO);   /* stdin  ребёнка = канал */
dup2(out, STDOUT_FILENO);  /* stdout ребёнка = файл  */
```

`dup2(old, new)` делает дескриптор `new` копией `old`, при необходимости предварительно закрывая `new`. Именно так выполняется **перенаправление ввода-вывода**: дочерняя программа не знает о каналах и файлах, она просто читает из дескриптора 0 и пишет в дескриптор 1. Важное свойство: копия, созданная `dup2`, **не** наследует `O_CLOEXEC`, поэтому переживёт `exec`.

4.9. Функция `execl`

Заменяет образ текущего процесса новой программой. Если вызов удался, управление в старый код никогда не возвращается; возврат означает ошибку. После неудачного `execl` в ребёнке вызывается `_exit(127)` (именно `_exit`, а не `exit`, чтобы не сбрасывать копии буферов stdio, унаследованные от родителя).

4.10. Функция `readlink` и путь к `child`

Файл `/proc/self/exe` — символическая ссылка на исполняемый файл текущего процесса. Функция `readlink` читает её значение, то есть путь вида `/.../build/bin/parent`. Затем `strrchr(buf,'/')` находит последний слэш, после него подставляется имя `child`. Так `parent` находит `child` в своей папке независимо от текущей рабочей директории.

4.11. Функция `write` и цикл `os_write_all`

`write` может записать меньше байт, чем запрошено (например, при заполненном канале или прерывании сигналом). Поэтому запись выполняется в цикле до тех пор, пока не будут переданы все данные. Код ошибки `EINTR` (вызов прерван сигналом) не считается ошибкой — запись просто повторяется.

4.12. Функция `waitpid`

```
1 waitpid(proc->pid, &status, 0);
```

Блокирует родителя до завершения указанного дочернего процесса и записывает в `status` причину завершения. Помимо получения кода возврата, вызов **освобождает запись о процессе в ядре**; пока родитель не вызвал `waitpid`, завершившийся ребёнок остаётся «зомби-процессом».

4.13. Макросы `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`

Значение `status` упаковано в битовое поле, поэтому расшифровывается макросами:

Макрос	Смысл
`WIFEXITED(s)`	Истина, если процесс завершился нормально (`return` из `main` или `exit`).
`WEXITSTATUS(s)`	Код возврата (имеет смысл, только если `WIFEXITED` истинно).
`WIFSIGNALED(s)`	Истина, если процесс был завершён сигналом (например, `SIGSEGV`, `SIGKILL`).
`WTERMSIG(s)`	Номер сигнала, убившего процесс.

4.14. Сигнал `SIGPIPE`

Если записать в канал, у которого не осталось читателей, ядро посылает процессу `SIGPIPE`, что по умолчанию убивает программу. В родителе вызывается `signal(SIGPIPE, SIG_IGN)`: тогда `write` просто вернёт ошибку `EPIPE`, которую можно обработать и корректно сообщить пользователю.

5. Платформонезависимые функции

5.1. `read_line`

Читает строку произвольной длины из `FILE*`. Буфер начинается с 128 байт и удваивается через `realloc`, когда заканчивается место. Это исключает типичную

ошибку фиксированного буфера (`char buf[256]`), из-за которой длинная строка обрезалась бы или портила память. Параметр `keep_nl` определяет, сохранять ли `\n`; символ `\r` перед `\n` (формат CRLF) отбрасывается. Ошибка передаётся через `*err`, а различие между «конец ввода» и «ошибка» делается проверкой `ferror`.

5.2. Инверсия строки в `child`

```
static void reverse(char *s, size_t len)
{
    size_t i, j;
    char tmp;
    if (len < 2) return;
    for (i = 0, j = len - 1; i < j; i++, j--) {
        tmp = s[i]; s[i] = s[j]; s[j] = tmp;
    }
}
```

Алгоритм «двух указателей»: один идёт с начала, другой с конца, символы меняются местами до встречи в середине. Проверка `len < 2` необходима, так как `len` имеет беззнаковый тип `size_t`, и выражение `len - 1` для пустой строки дало бы огромное число. После каждой строки в `child` вызывается `fflush(stdout)`, чтобы данные попадали в файл сразу, а не копились в буфере.

5.3. Обработка ошибок

`err_errno` печатает сообщение вида `<что делали>: <strerror(errno)>`. `err_sys` в Linux вызывает `err_errno`. Все функции слоя `os` возвращают `0` или `-1`, а сообщение об ошибке выводится на месте возникновения.

6. Алгоритм работы родительского процесса

1. Определить путь к `child` (`os_sibling_path`).
2. Прочитать два имени файлов.
3. Открыть файлы на запись и создать два канала.
4. Запустить `child1` (`stdin` = `pipe1`, `stdout` = файл 1). Закрыть в родителе читающий конец канала и дескриптор файла 1.
5. Запустить `child2` аналогично, закрыть его читающий конец и файл 2.
6. В цикле читать строки из `stdin`: нечётные записывать в `pipe1`, чётные в `pipe2`; при отсутствии завершающего `\n` добавлять его.
7. По концу ввода (Ctrl+D) закрыть записывающие концы каналов — дети получат EOF.
8. Дождаться завершения обоих детей и проверить их коды возврата.

Весь код очистки реализован через метку `cleanup` и `goto`: независимо от места возникновения ошибки выполняется один и тот же набор закрытий и ожиданий, что предотвращает утечки дескрипторов. Отдельные «закрыть в родителе» после запуска ребёнка нужны, чтобы в родителе не осталось лишних копий дескрипторов (иначе канал не получит EOF).

- **Ошибка `CMakeCache.txt directory is different`.** Папка `build` была создана CMake в другом окружении (другие пути и генератор), а затем использовалась из WSL (путь `/mnt/c/...`). Кэш конфигурации привязан к абсолютным путям и генератору, поэтому после смены окружения папку `build` нужно удалить и создать заново.
- **Ошибка `CMAKE_C_COMPILER not set`.** В WSL не были установлены компилятор и `make`. Решение: `sudo apt install build-essential cmake`.

7. Выводы

В ходе работы были изучены и применены на практике:

1. **Модель процессов.** Освоена пара `fork`/`exec` в Linux; понято, что создание процесса (`fork`) и запуск в нём новой программы (`exec`) являются двумя отдельными шагами.
2. **Неименованные каналы.** Канал — односторонний буфер в ядре. Однонаправленность и семантика EOF (конец данных наступает только когда закрыты *все* записывающие концы) определяют правильное закрытие дескрипторов в родителе и ребёнке.
3. **Перенаправление потоков.** С помощью `dup2` стандартные потоки ребёнка подменяются каналом и файлом, поэтому `child` является обычной консольной программой и не зависит от способа запуска.
4. **Наследование дескрипторов.** Утечка дескрипторов в дочерний процесс — частая причина зависаний. Для её предотвращения применяется флаг `O_CLOEXEC`.
5. **Ожидание процессов.** Вызов `waitpid` необходим, чтобы получить результат работы ребёнка и не оставлять зомби-процессов. Разбор статуса через `WIFEXITED`/`WIFSIGNALED` позволяет отличить нормальное завершение от аварийного.
6. **Надёжный ввод-вывод.** `write` может записать не все данные, а системные вызовы могут быть прерваны сигналом (`EINTR`), поэтому используется цикл записи и повтор вызова. Чтение строк произвольной длины реализовано динамическим буфером.
7. **Модульность.** Разделение кода на общий слой, интерфейс `os.h` и его реализацию `os_linux.c` позволило отделить логику родителя от системных вызовов и упростить чтение и отладку программы.
8. **Инструменты.** Освоена система сборки CMake (статические библиотеки, зависимости целей, `ctest`), а также особенности запуска проекта в WSL и проблема кэша CMake при смене среды сборки.

Практически важный вывод: большинство ошибок при работе с процессами и каналами связано не с самими вызовами, а с **владением дескрипторами** — кто и когда закрывает какой конец канала. Именно аккуратное закрытие лишних копий обеспечивает корректное завершение программы.

Личные результаты. В ходе выполнения лабораторной работы я научился создавать процессы с помощью системного вызова `fork` и запускать в них

другие программы через `exec`. Я разобрался, как родительский процесс управляет дочерними: передаёт им данные через неименованные каналы (`pipe`), перенаправляет их ввод и вывод с помощью `dup2`, ожидает завершения через `waitpid` и по статусу определяет, завершился процесс нормально или был убит сигналом. Также я понял, как важно закрывать ненужные дескрипторы и как работают наследование дескрипторов, конец файла (EOF) в канале и сигналы (например, `SIGPIPE` и `SIGSEGV`). Кроме того, я научился находить и исправлять ошибки при работе с процессами, в том числе падение дочернего процесса из-за неподключённых заголовочных файлов.